Resumen Analítico: Módulo 4 de Python

1. Manipulación de Archivos TXT

Concepto: Apertura y Gestión Segura de Archivos

- **Pregunta**: ¿Cómo se abre un archivo en Python y cuál es la manera más segura de gestionarlo?
- **Evidencia**: Se utiliza la función `open()`, la cual retorna un objeto `_io.TextIOWrapper`. Al combinarlo con el bloque `with open(...) as f:`, se evita el uso manual de `f.close()`.
- **Conclusión**: El gestor de contexto `with` es el estándar de la industria porque previene fugas de memoria, cerrando el archivo de manera automática incluso si ocurre un error.

Concepto: Modos de Escritura y Codificación

- **Pregunta**: ¿Qué función cumplen los parámetros `mode` y `encoding` al interactuar con un documento?
- **Evidencia**: El modo `w` (write) crea o sobrescribe un archivo entero, mientras que `a` (append) preserva el contenido y añade datos al final. El parámetro `encoding` (como utf-8) indica el esquema de caracteres.
- **Conclusión**: Especificar correctamente el modo evita la pérdida accidental de datos, y fijar el encoding previene errores de lectura de caracteres especiales.

Concepto: Inserción de Contenido

- **Pregunta**: ¿Cuál es la diferencia entre los métodos `write()` y `writelines()`?
- **Evidencia**: `write()` inserta una única cadena de texto. Por el contrario, `writelines()` toma una lista de cadenas, pero requiere que el programador incluya explícitamente los saltos de línea (`\n`) en cada elemento.
- **Conclusión**: Conocer la naturaleza del dato entrante (una sola variable de texto o una lista iterable) dicta qué función utilizar para la inserción masiva o individual.

2. Archivos Estructurados: CSV y JSON

Concepto: Procesamiento Tabular (CSV)

- **Pregunta**: ¿Cómo transforma Python los datos tabulares delimitados por comas u otros caracteres?
- **Evidencia**: Mediante el módulo `csv`. La función `csv.reader` lee el archivo línea por línea devolviendo una lista de strings para cada fila, mientras que `csv.writer` permite volcar listas a disco (usando `writerows`).
- **Conclusión**: Python abstrae el análisis manual de los delimitadores, haciendo que procesar y generar hojas de cálculo básicas sea extremadamente veloz e intuitivo.

Concepto: Jerarquía de Datos (JSON)

- **Pregunta**: ¿De qué manera facilita Python el intercambio de datos web en formato JSON?
- **Evidencia**: Con la importación del módulo nativo `json`. La función `json.load()` se encarga de leer el fichero de texto y de parsear la estructura anidada.
- **Conclusión**: Dado que JSON y los diccionarios de Python comparten una filosofía estructural casi idéntica, la conversión genera directamente diccionarios (`dict`) y listas (`list`) listas para ser consumidas.

3. Interacción con el Sistema Operativo (Módulo OS)

Concepto: Navegación del Sistema de Archivos

- **Pregunta**: ¿Qué herramientas otorga el módulo `os` para orientar y analizar rutas en el disco?
- **Evidencia**: `os.getcwd()` devuelve el directorio de trabajo activo (CWD). A su vez, `os.listdir()` genera una lista del contenido de un directorio, y `os.path.join()` ensambla rutas de manera compatible para cualquier sistema operativo.
- **Conclusión**: Construir rutas con `os.path.join()` es imperativo para garantizar que el código se ejecute sin modificaciones tanto en Windows como en entornos Unix.

Concepto: Validación de Elementos

- **Pregunta**: ¿Cómo discrimina un script entre archivos, carpetas y objetos inexistentes?

- **Evidencia**: A través de las funciones booleanas `os.path.isfile()` y `os.path.isdir()`, las cuales evalúan la ruta retornando `True` o `False`.
- **Conclusión**: Estas validaciones previas son cruciales para crear lógica condicional, evitando fallos en tiempo de ejecución al intentar manipular algo que no existe.

Concepto: Gestión de Carpetas y Ficheros

- **Pregunta**: ¿De qué modo se simulan las acciones clásicas del explorador de archivos desde código?
- **Evidencia**: Se usa `os.mkdir()` para instanciar carpetas, `os.rmdir()` para borrarlas (solo si están vacías) y `os.remove()` para eliminar archivos. Adicionalmente, `os.rename()` permite modificar el nombre y/o mover el archivo.
- **Conclusión**: El módulo otorga autonomía absoluta al script para que administre su propio entorno de trabajo.

Concepto: Consola de Sistema Integrada

- **Pregunta**: ¿Cómo se extienden las capacidades de Python para ejecutar acciones nativas del sistema operativo huésped?
- **Evidencia**: Se invoca a `os.system("comando")`, lo que instruye al intérprete de comandos local (CMD, bash, etc.) a ejecutar una instrucción externa (como `cls` o invocar otro script).
- **Conclusión**: Actúa como un puente de automatización, permitiendo a Python delegar tareas específicas a las utilidades propias de Windows o Linux.